

```

int read_cnt;

/* Read the data from the bulk endpoint */
retval = usb_bulk_msg(device, usb_rcvbulkpipe(device, BULK_
EP_IN),
    bulk_buf, MAX_PKT_SIZE, &read_cnt, 5000);
if (retval)
{
    printk(KERN_ERR "Bulk message returned %d\n", retval);
    return retval;
}
if (copy_to_user(buf, bulk_buf, MIN(cnt, read_cnt)))
{
    return -EFAULT;
}

return MIN(cnt, read_cnt);
}

static ssize_t pen_write(struct file *f, const char __user *buf,
size_t cnt, loff_t *off)
{
    int retval;
    int wrote_cnt = MIN(cnt, MAX_PKT_SIZE);

    if (copy_from_user(bulk_buf, buf, MIN(cnt, MAX_PKT_SIZE)))
    {
        return -EFAULT;
    }

    /* Write the data into the bulk endpoint */
    retval = usb_bulk_msg(device, usb_sndbulkpipe(device, BULK_
EP_OUT),
        bulk_buf, MIN(cnt, MAX_PKT_SIZE), &wrote_cnt, 5000);
    if (retval)
    {
        printk(KERN_ERR "Bulk message returned %d\n", retval);
        return retval;
    }

    return wrote_cnt;
}

static struct file_operations fops =
{
    .open = pen_open,
    .release = pen_close,
    .read = pen_read,
    .write = pen_write,
};

static int pen_probe(struct usb_interface *interface, const
struct usb_device_id *id)
{

```

```

    int retval;

    device = interface_to_usbdev(interface);

    class.name = "usb/pen%d";
    class.fops = &fops;
    if ((retval = usb_register_dev(interface, &class)) < 0)
    {
        /* Something prevented us from registering this driver */
        err("Not able to get a minor for this device.");
    }
    else
    {
        printk(KERN_INFO "Minor obtained: %d\n", interface-
>minor);
    }

    return retval;
}

static void pen_disconnect(struct usb_interface *interface)
{
    usb_deregister_dev(interface, &class);
}

/* Table of devices that work with this driver */
static struct usb_device_id pen_table[] =
{
    { USB_DEVICE(0x058F, 0x6387) },
    {} /* Terminating entry */
};
MODULE_DEVICE_TABLE(usb, pen_table);

static struct usb_driver pen_driver =
{
    .name = "pen_driver",
    .probe = pen_probe,
    .disconnect = pen_disconnect,
    .id_table = pen_table,
};

static int __init pen_init(void)
{
    int result;

    /* Register this driver with the USB subsystem */
    if ((result = usb_register(&pen_driver)))
    {
        err("usb_register failed. Error number %d", result);
    }
    return result;
}

```